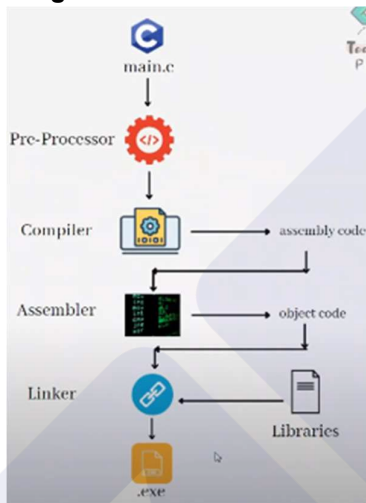


Compilation Process of a C Program

The compilation is the process of converting the source code of the C language into **machine code**. As C is a **mid-level programming language**, it needs a compiler to convert it into an **executable code** so that the program can run on a computer. A compiler converts a C program into an executable in **four main phases**:

Phases of Compilation

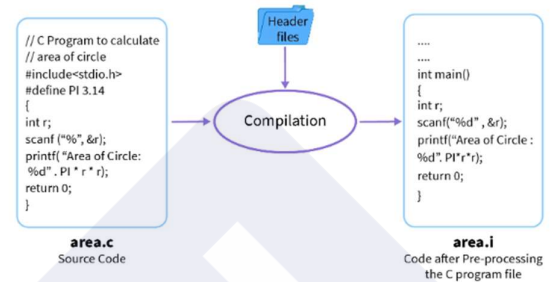
1. **Pre-processing**
2. **Compilation**
3. **Assembly**
4. **Linking**



1. Pre-processing

- The preprocessor is the **first stage** of the compilation process.
- It handles preprocessor directives (statements beginning with #).
- Major tasks:
 1. **Removal of comments** (since comments are only for human understanding).
 2. **Expansion of macros** defined with #define.
 3. **Expansion of included header files** (#include <stdio.h> etc.).
 4. **Conditional compilation** using #ifdef, #ifndef, etc.

Output: An intermediate file with extension .i that contains expanded source code.



Example:

If we write:

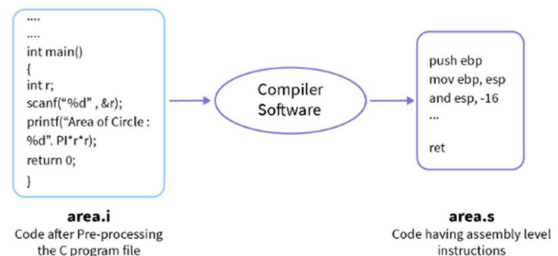
```
#include <stdio.h>
#define PI 3.14
```

After preprocessing, the file .i will contain the actual code of <stdio.h> and wherever PI is written, it will be replaced with 3.14.

2. Compilation

- The compiler takes the preprocessed file (.i) and **translates it into assembly language** (.s).
- **Activities:**
 - **Lexical analysis** → breaks program into tokens.
 - **Syntax analysis** → checks grammar rules.
 - **Semantic analysis** → ensures logical correctness (e.g., type checking).
 - **Optimization** → improves performance and efficiency.

Output: Assembly code file with extension .s.



Example:

A C statement like:

```
int a = 5 + 3;
```

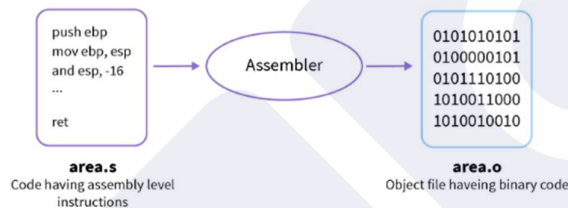
Might be translated into assembly instructions such as:

```
MOV R1, #5
ADD R1, R1, #3
MOV a, R1
```

3. Assembly

- The assembler converts the **assembly code (.s)** into **object code (.o/.obj)**.
- Object code is machine-level code but may still contain unresolved references to external libraries.

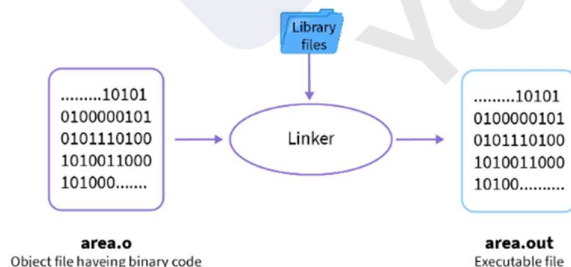
Output: Object file with extension .o (UNIX) or .obj (Windows).



4. Linking

- The linker combines the object file with necessary **library files**.
- It resolves **external function calls** (e.g., printf() from <stdio.h>).
- Produces the final **executable file** (.exe in DOS/Windows, a.out in UNIX/Linux).

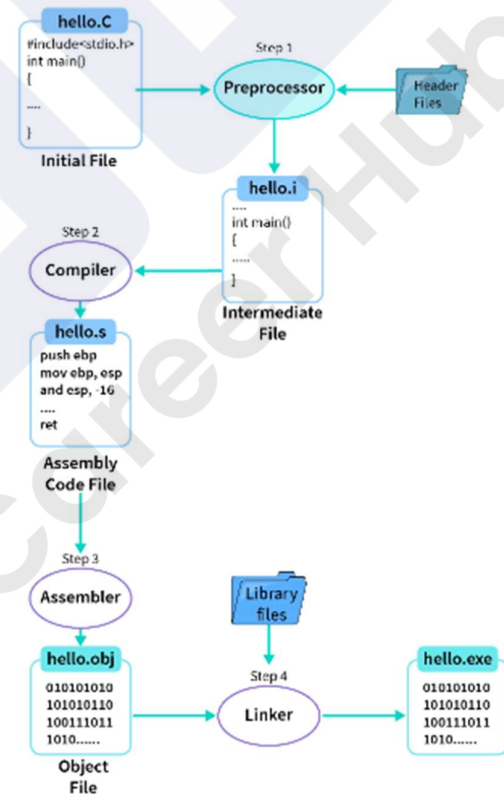
Example:
If you use printf("Hello"); in your program, the actual machine code for printf is taken from the **C Standard Library** during linking.



Example program to show the compilation process via diagram

```
// Simple Hello World program in C
#include <stdio.h>
int main()
{
    // printf() is an output function which prints
    // the passed string in the output console
    printf("Hello World!");
    return 0;
}
```

Flow Diagram of the Program



Role of the Pre-Processor in the compilation process

A Preprocessor is a system software (a computer program that is designed to run on computer's hardware and application programs). It performs preprocessing of the High Level Language(HLL). Preprocessing is the first step of the language processing system. Language processing system translates the high level language to machine level language or absolute machine code(i.e. to the form that can be understood by machine).

The preprocessor doesn't know about the scope rules of C. Preprocessor directives like #define come into effect as soon as they are seen and remain in effect until the end of the file that contains them; the program's block structure is irrelevant

A Preprocessor mainly performs three tasks on the HLL code :

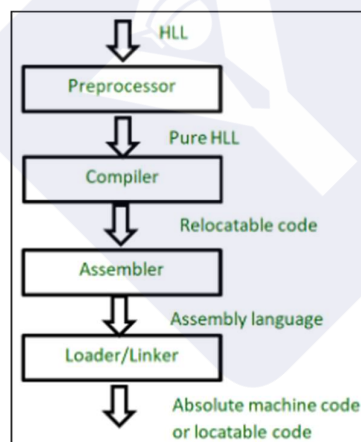
- ❖ **Removing comments** : It removes all the comments. A comment is written only for the humans to understand the code. So, it is obvious that they are of no use to a machine. So, preprocessor removes all of them as they are not required in the execution and won't be executed as well.
- ❖ **File inclusion** : Including all the files from library that our program needs. In HLL we write #include which is a directive for the preprocessor that tells it to include the contents of the library file specified. For example, #include will tell the preprocessor to include all the contents in the library file stdio.h.
- ❖ **Macro expansion** : Macros can be called as small functions that are not as overhead to process. If we have to write a function (having a small definition) that needs to be called recursively (again and again), then we should prefer a macro over a function.

Basic Structure of C Program or Main components of a basic C program

The basic structure of a C program is divided into 6 parts which makes it easy to read, modify, document, and understand in a particular format.

Sections of the C Program

There are 6 basic sections responsible for the proper execution of a program. Sections are mentioned below:



1. Documentation
2. Preprocessor Section
3. Definition
4. Global Declaration
5. Main() Function
6. Sub Programs

1. Documentation

This section consists of the description of the program, the name of the program, and the creation date and time of the program. It is specified at the start of the program in the form of comments. Documentation can be represented as:

Example:

```
// description, name of the program, programmer name, date, time etc.  
or  
/*  
description, name of the program, programmer name, date, time etc.  
*/
```

Anything written as comments will be treated as documentation of the program and this will not interfere with the given code. Basically, it gives an overview to the reader of the program.

2. Preprocessor Section

All the header files of the program will be declared in the preprocessor section of the program. Header files help us to access other's improved code into our code. A copy of these multiple files is inserted into our program before the process of compilation.

Example:

```
#include<stdio.h>  
#include<math.h>
```

3. Definition

Preprocessors are the programs that process our source code before the process of compilation. There are multiple steps which are involved in the writing and execution of the program. Preprocessor directives start with the '#' symbol. The #define preprocessor is used to create a constant throughout the program. Whenever this name is encountered by the compiler, it is replaced by the actual piece of defined code.

Example:

```
#define Pi 3.14
```

4. Global Declaration

The global declaration section contains global variables, function declaration, and static variables. Variables and functions which are declared in this scope can be used anywhere in the program.

Example:

```
int num = 18;
```

5. Main() Function

Every C program must have a main function. The main() function of the program is written in this

section. Operations like declaration and execution are performed inside the curly braces of the main program. The return type of the main() function can be int as well as void too. void() main tells the compiler that the program will not return any value. The int main() tells the compiler that the program will return an integer value.

Example:

```
int main(){  
    return 0;  
}
```

6. Sub Programs

User-defined functions are called in this section of the program. The control of the program is shifted to the called function whenever they are called from the main or outside the main() function. These are specified as per the requirements of the programmer.

```
int sum(int x, int y)  
{  
    return x+y;  
}
```

Example program to show a complete structure of C Language

```
// Documentation  
/**  
 * file: sum.c  
 * author: you  
 * description: program to find sum.  
 */  
  
// Link  
#include <stdio.h>  
  
// Definition  
#define X 20  
  
// Global Declaration  
int sum(int y);  
  
// Main() Function  
int main(void)  
{  
    int y = 55;  
    printf("Sum: %d", sum(y));  
    return 0;  
}  
  
// Subprogram  
int sum(int y)  
{  
    return y + X;  
}
```

Explanation of the above Program

Below is the explanation of the above program. With a description explaining the program's meaning and use.

```
// Documentation  
/**  
 * file: sum.c  
 * author: you  
 * description: program to find sum.  
 */
```

It is the comment section and is part of the description section of the code.

```
// Link  
#include <stdio.h>
```

Header file which is used for standard input-output. This is the preprocessor section.

```
// Definition  
#define X 20
```

This is the definition section. It allows the use of constant X in the code.


```
// Global Declaration  
int sum(int y);
```

This is the Global declaration section that includes the function declaration that can be used anywhere in the program.

```
// Main() Function  
int main(void)  
{  
    int y = 55;  
    printf("Sum: %d", sum(y));  
    return 0;  
}
```

main() is the first function that is executed in the C program.

{...} These curly braces mark the beginning and end of the main function.

printf("Sum: %d", sum(y)); printf() function is used to print the sum on the screen.

return 0; We have used int as the return type so we have to return 0, which states that the given program is free from error and it can be exited successfully.

```
// Subprogram  
int sum(int y)  
{  
    return y + X;  
}
```

This is the subprogram section. It includes the user-defined functions that are called in the main() function.

Importance of header files in a C program.

Header files in C play a crucial role in organizing and managing large programs by providing a way to share information between different parts of a program. They contain function declarations, macro definitions, constants, and data type definitions that can be used across multiple source files. Here's a detailed explanation of their importance:

1. Function Declarations (Prototypes)

Header files contain function declarations (also called prototypes), which inform the compiler about the functions used in different source files. This allows the compiler to check for correct function usage, even if the function's definition is in a different file.

```
// myheader.h  
int add(int a, int b); // Function prototype
```

2. Code Reusability

By placing function prototypes, macros, and definitions in a header file, you can reuse the same code across multiple source files without rewriting it. This promotes modularity and cleaner code organization.

Example: Including a custom header file in multiple source files

```
#include "myheader.h"
```

3. Library Function Access

Standard C library functions, such as printf(), scanf(), malloc(), etc., are declared in standard header files (e.g., stdio.h, stdlib.h). By including these header files, you gain access to built-in library functions without needing to know their internal implementation.

```
#include <stdio.h> // Allowing access to printf() and other functions
```

4. Macro Definitions

Header files allow you to define macros, which are constants or code snippets that are replaced during preprocessing. These macros can be used throughout your program, making the code easier to maintain and update.

```
#define PI 3.14159 // Macro definition
```

5. Data Type Definitions

Custom data types, such as struct or typedef, can be defined in header files. This ensures that these types are available to any source file that includes the header.

```
// mytypes.h
typedef struct {
    int x;
    int y;
} Point;
```

Importance of comments in a c program

Comments are an essential part of any programming language, including C, as they help improve the readability, maintainability, and understanding of the code. Here are the key reasons why comments are important in a C program:

1. Improves Code Readability

- Purpose: Comments describe what the code is doing in plain language, making it easier for anyone reading the code (including the original author) to understand its purpose and logic.

```
// Calculate the area of a circle
area = PI * radius * radius;
```

2. Facilitates Code Maintenance

- Purpose: Comments help in maintaining and updating code over time. Developers working on the program in the future can quickly understand the logic behind various sections, making bug fixing or enhancements easier.

3. Explains Complex Logic

- Purpose: Some sections of code can involve complex algorithms or logic that might not be immediately clear to a reader. Comments can clarify how and why certain decisions were made or explain the implementation of an algorithm.

4. Provides Context for Future Modifications

```
/* This program calculates
the factorial of a number
using a recursive function */

int factorial(int n) {
    if (n == 0) { // Base case
        return 1;
    } else {
        return n * factorial(n - 1); // Recursive
        case
    }
}
```

- Purpose: Comments serve as a guide for future modifications. They can explain why certain design decisions were made, helping future developers or even the original author understand the reason behind a particular implementation.

5. Serves as In-line Documentation

- Purpose: Comments act as in-line documentation, especially when used to provide quick explanations for particular lines or blocks of code, making it easier to trace the program flow.

```
int sum = 0; // Initialize sum to zero
```

Types of comments in C

In C, there are two types of comments:

1. Single-line Comments

Syntax: Begins with // and extends to the end of the line.

Usage: Single-line comments are typically used for brief descriptions or explanations of specific lines or sections of code. They are ideal for short comments that don't require multiple lines.

```
int x = 10; // Initialize variable x to 10
```

In the example above, the comment // Initialize variable x to 10 explains what the line of code is doing. Anything after // on the same line is considered a comment and will not be executed by the compiler.

2. Multi-line Comments

Syntax: Starts with /* and ends with */. Everything between these markers is treated as a comment, regardless of how many lines it spans.

Usage: Multi-line comments are used when you need to explain a larger section of code or provide more detailed information. They can span multiple lines, making them useful for longer

descriptions or for temporarily disabling blocks of code.

In this example, the multi-line comment explains the purpose of the program. It spans several lines, making it easier to read when a more detailed explanation is needed.

Macros in C

The macro in C language is known as the piece of code which can be replaced by the macro value. The macro is defined with the help of `#define` preprocessor directive and the macro doesn't end with a semicolon(`;`). Macro is just a name given to certain values or expressions it doesn't point to any memory location.

Whenever the compiler encounters the macro, it replaces the macro name with the macro value. Two macros could not have the same name.

The syntax of the macro is as shown in the following figure. Here, we will have the three components:

1. `#define` - Preprocessor Directive
2. `PI` - Macro Name
3. `3.14` - Macro Value



Types of Macros in C Language

The macro in C language is classified based on the different types of values that it replaces.

Let's see the different types of macros one by one.

1. Object like Macros in C

An object like macros in C programming is simply the macros that get replaced by certain values or segments of code.

In the above example, in the introduction, we saw the macro with the name `PI` and value `3.14` it is an example of an object like macros. We can define macros with different datatypes as shown as follows but we can't use the same macro names for two different values –

// Examples of object-like macros in C language

```
#define MAX 100
#define MIN 1
#define GRAVITY 9.8
#define NAME "Scaler"
#define TRUE 1
#define FALSE 0
```

2. Function Like Macros in C

- In the function like macros are very similar to the actual function in C programming.
- We can pass the arguments with the macro name and perform the actions in the code segment.
- In macros, there is no type checking of arguments so we can use it as an advantage to pass different datatypes in same macros in C language.

Let's consider the following code example which is

```
#include <stdio.h>
// Object-like macro
#define PI 3.14
// Function-like macro
#define Area(r) (PI*(r*r))
void main()
{
    // Declaration and initialization of radius
    float radius = 2.5;

    // Declaring and assigning the value to area
    float area = Area(radius);

    // Printing the area of circle
    printf("Area of circle is %f\n", area);

    // Using radius as int data type
    int radiusInt = 5;
    printf("Area of circle is %f", Area(radiusInt));
}
```

C keywords

Keywords are predefined or reserved words that have special meanings to the compiler. These are part of the syntax and cannot be used as identifiers in the program. A list of keywords in C or reserved words in the C programming language are mentioned below:

Importance of C Keywords:

Defines the Structure of the Language: Keywords are the fundamental building blocks of C programs. They are used to define the structure and control flow of the program, such as loops, conditional statements, and function declarations.

auto double int struct break
else long switch case enum
register typedef char extern return
union continue for signed void
do if static while default
goto sizeof volatile const float
short unsigned

Predefined Meaning: Each keyword has a specific meaning, which the C compiler interprets. For instance, the keyword `int` is used to declare integer variables, while `if` is used for conditional branching.

Controls Program Flow: Keywords like `if`, `for`, `while`, `switch`, and `return` are used to control the logical flow and decision-making in a program. Without them, it would be impossible to manage how a program runs.

Data Types and Memory Management: Keywords such as `int`, `float`, `char`, and `double` are used to define the types of data that variables can store. Others like `sizeof` and `return` are used for memory management and returning values from functions.

Example Usage:

```
#include <stdio.h>
int main() {
    int a = 10;      // 'int' is a keyword for
                    // declaring an integer variable

    if (a > 5) {      // 'if' is a keyword for
                    // conditional branching
        printf("a is greater than 5\n");
    }

    return 0;        // 'return' is used to return a
                    // value from the function
}
```

In this example, the keywords `int`, `if`, and `return` are used to declare a variable, control program flow, and return a value, respectively.

Tokens in C Programming

In C programming, tokens are the smallest elements or building blocks of a program. The compiler uses these tokens to interpret and process the source code. Tokens represent the fundamental syntax used to write any C program.

Types of Tokens in C:

There are 6 types of tokens in C:

Keywords

These are reserved words that have a predefined meaning in the language. They cannot be used as identifiers (e.g., variable names).

Example: `int`, `return`, `if`, `while`, `for`, `void`.

Identifiers

Identifiers are names given to various program elements like variables,

functions, arrays, and structures. They are user-defined names.

Rules: Identifiers must start with a letter (a-z, A-Z) or an underscore (`_`), followed by letters, digits (0-9), or underscores.

They are case-sensitive.

Example: `sum`, `main`, `totalMarks`, `myFunction`.

Constants

Constants are fixed values that cannot be changed during the execution of a program. These can be numeric values, characters, or strings.

Types:

1. Integer constants: e.g., 10, -20
2. Floating-point constants: e.g., 3.14, -0.001

3. Character constants: e.g., 'A', 'Z'
4. String literals: e.g., "Hello, World!"

Operators

Operators are symbols used to perform operations on variables and values. These can be arithmetic, relational, logical, or bitwise operations.

Categories:

1. Arithmetic operators: +, -, *, /, %
2. Relational operators: ==, !=, >, <, >=, <=
3. Logical operators: &&, ||, !
4. Assignment operators: =, +=, -=, etc.
5. Bitwise operators: &, |, ^, ~, <<, >>

Strings

Strings are sequences of characters enclosed within double quotes ("). Strings are essentially arrays of characters in C.

Example: "Hello", "C Programming".

Example Program with Tokens:

```
#include <stdio.h> // Keyword: #include, Identifier: stdio.h

int main() { // Keyword: int, Identifier: main, Punctuators: (), {}
    int a = 10, b = 20; // Keywords: int, Identifiers: a, b, Constants: 10, 20
    int sum = a + b; // Operators: =, +, Identifiers: sum, a, b
    printf("Sum is %d\n", sum); // String: "Sum is %d\n", Identifiers: printf, sum
    return 0; // Keyword: return, Constant: 0
}
```

Breakdown of Tokens in the Example:

- ❖ **Keywords:** int, return
- ❖ **Identifiers:** main, a, b, sum, printf
- ❖ **Constants:** 10, 20, 0
- ❖ **Operators:** =, +
- ❖ **Punctuators:** (), {}, :, ,
- ❖ **Strings:** "Sum is %d\n"

Identifiers in C Programming

Identifiers in C programming are names used to identify variables, functions, arrays, structures, or any other user-defined item in a program. They allow the programmer to refer to and manipulate different elements in the code.

Characteristics of Identifiers:

- **User-defined:** Unlike keywords, which have special meaning in C, identifiers are created by the programmer.
- **Meaningful Names:** Identifiers are typically chosen to give meaningful names to program elements, making the code easier to understand and maintain.

Rules for Naming Identifiers:

- Start with a letter or **underscore (_)**: Identifiers must begin with either a letter (**a-z or A-Z**) or an underscore (_). They cannot start with a digit.
- **Valid:** myVar, _index
- **Invalid:** 1stValue, 9result
- Can contain letters, digits, and underscores: After the first character, the identifier can consist of letters, digits, or underscores.
- **Valid:** var_123, num1, count_
- **Invalid:** my@var, hello!
- Case-sensitive: Identifiers in C are case-sensitive, meaning that **MyVar** and **myvar** are treated as different identifiers.
- `int myvar = 10;`
- `int MyVar = 20;` These two variables are considered separate.

No spaces or special characters: Spaces or special characters (such as @, #, !, \$, etc.) are not allowed in identifiers.

Invalid: my var, my\$var

Cannot be a keyword: Identifiers cannot have the same name as any of C's reserved keywords like **int**, **return**, **for**, **while**, etc.

Examples of Valid and Invalid Identifiers:

- **Valid Identifiers:**
 - age, num_students, max_value, Score1, _total
- **Invalid Identifiers:**
 - 2ndItem (starts with a digit)
 - my var (contains a space)
 - char* (contains a special character)

Example of Identifiers in a Program:

```
#include <stdio.h>

int main() {
    int count = 5;    // 'count' is an
    identifier
    float average = 4.5; // 'average' is an
    identifier

    printf("Count: %d\n", count);
    printf("Average: %.2f\n", average);

    return 0;        // 'main', 'printf', and
    'return' are identifiers
}
```

Explain the difference between identifier and keywords

Aspect	Identifiers	Keywords
Definition	User-defined names for variables, functions, arrays, etc.	Reserved words that have a special meaning in C
Who Defines?	Defined by the programmer	Predefined by the C language
Purpose	Used to name elements such as variables, functions, arrays, structures, etc.	Used to specify commands or syntax in the language
Example	sum, age, myFunction, totalMarks	int, return, if, while, for
Modifiable?	Can be modified or created by the programmer	Cannot be modified, and their meaning is fixed
First Character Rule	Must begin with a letter or underscore (_)	No such rule; keywords are predefined
Case Sensitivity	Case-sensitive (e.g., Sum and sum are different)	Case-sensitive (e.g., if and If are different)